

LN6. 最小生成树 (Minimum Spanning Tree, MST)

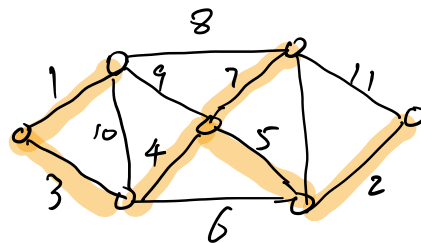
* 定义:

已知: 图 $G=(V, E)$, 无向, 简单, 连通图.

$\forall e \in E, C_e > 0$. 带权图.

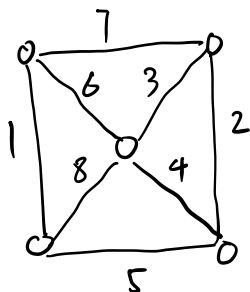
求: 连通子图 $V(T) = V$

目标: 最小化 $C(T) = \sum_{e \in E(T)} C_e$

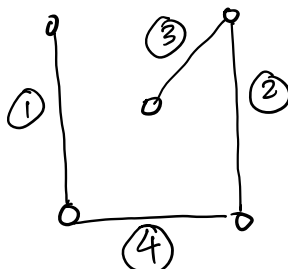


* 性质: 无环, 连通 $\Rightarrow$ 树.

* 算法:

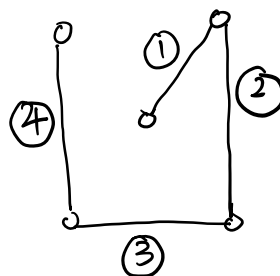


Kruskal:



保持无环.
[kill circle]

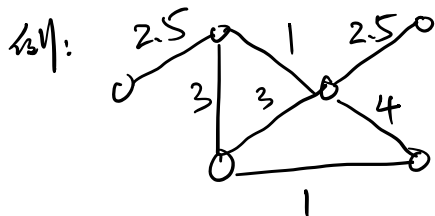
Prim:



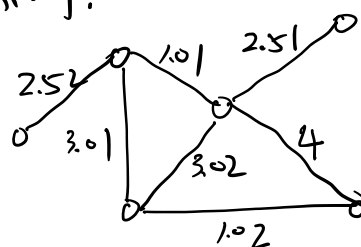
保持连通.
不断增长树.

* 算法的正确性证明:

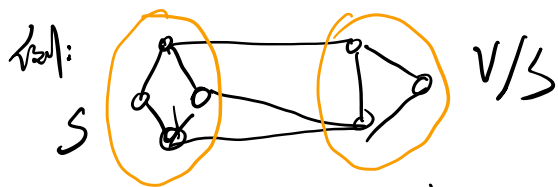
假设: 不失一般性, 假设边权均不同.



$\Rightarrow$

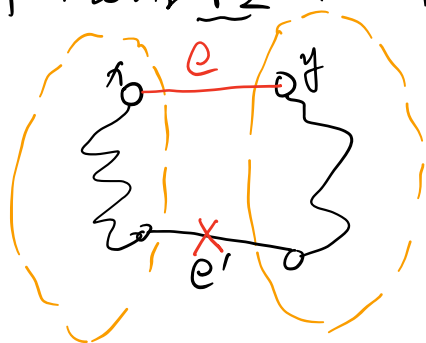


Def. 割边 (cut): $S \neq \emptyset$, S 与 V/S 之间的边构成的集合.



Lemma. MST 的割边性质:
 对于任意一个割 $(S, V/S)$, 其割边中最小权的边 e
 一定被包含在 MST 中.

Pf. 反证: 若 T 是一个 MST, 但不包含 e . 则一定存在另一条割边 e'
 $C_e < C_{e'}$



加入 e , 去掉 e' . 得到新的 ST, T'

$$C_{(T-e)} + C_e - C_{e'} \Rightarrow C_{T'}$$

$$C_{T'} < C_T \quad \text{矛盾.}$$

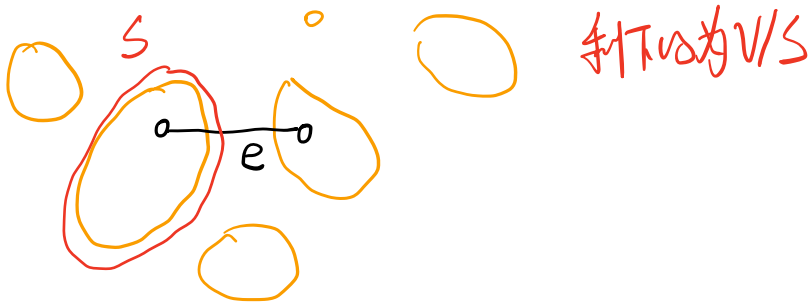
$\therefore$ 最小割边 e 一定在 MST 中.

Thm. Kruskal 算法输出的是 MST.

Pf: ① ST

② 每次加入的边均满足以上 lemma.

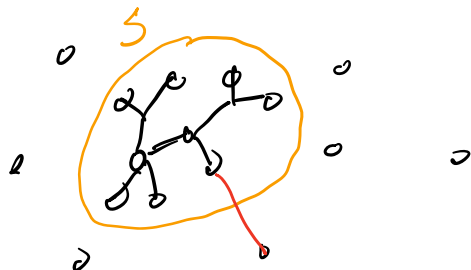
即一定且 割边 中的最小权的边.



Thm. Prim 算法输出的是 MST.

Pf. ① ST

② 加入的边均满足以上 Lemma.

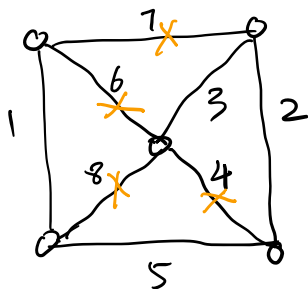


其子结构的集合为 V/S .

LN7. 克鲁斯卡尔 MST 算法.

Alg3. 减边算法: 从 G 出发.

每一步, 在保持连通性的前提下,
删去权最大的边.



Q: 如何证明: hw.

Alg4. Blue / Red 算法.

- 循环 {
- ① Blue, 加边: 找一个 cut (X, Y) , $X \leftrightarrow Y$ 无蓝边. 选此 cut 中最小权的边加入. 染为 蓝色.
 - ② Red, 减边: 找一个 cycle C , 上面无红边. 选此环中最大权的边, 染为 红色.

直到所有边被染色.

输出 蓝边集为 MST.

LN8. 降低MST算法的复杂度 $\Rightarrow$ 基于并查集的MST.

* Kruskal算法的复杂度分析: $|V|=n, |E|=m$

$T = (V, \phi)$

循环. while T 不连通.

$O(n)$, $n-1$ 次.

选择连接2个子树的最优边 e . $O(m)$

$T = T \cup \{e\}$.

$O(1)$

end

返回 T .

$\Rightarrow$ 复杂度 $O(mn)$

* 对Kruskal算法. 如何降低复杂度? 排序.

对边从小到大排序

$O(m \log n)$

循环: 对排序的边 $e = (u, v)$

m 次

if u, v 属于不同的连通子图. $\leftarrow \text{Find}(u), \text{Find}(v)$

$\log n$

$T = T \cup \{e\}$

更新 T 的连通子图.

$\leftarrow \text{Union}(T(u), T(v))$

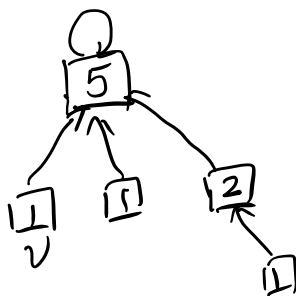
$\log n$

end if

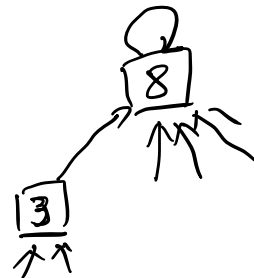
end for

输出 T .

*



Union $\rightarrow$



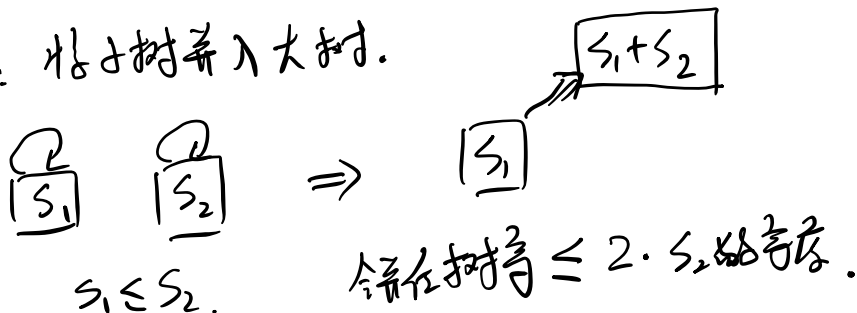
初始化: n 个节点, 每个是 i



$\text{Find}(u)$: 从 u 出发 找到树根 输出根节点 $O(\log n)$

$\text{Union}(u, v)$: $r_1 \leftarrow \text{Find}(u)$ $r_2 \leftarrow \text{Find}(v)$ $O(\log n)$
把小树并入大树, 合并后根节点处 树的大小.

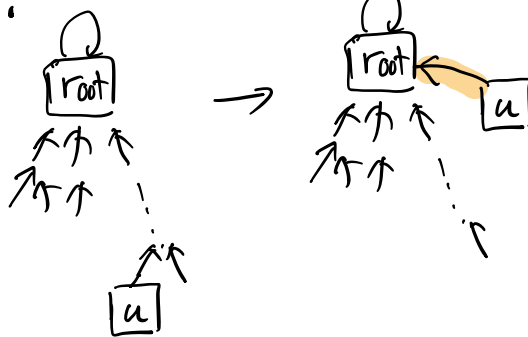
关键: 将小树并入大树.



$\Rightarrow$ 算法复杂度为 $O(m \log n)$.

* 计算 MST 的更快的算法?

路径压缩: $\text{Find}(u)$



补充: 逆 Ackermann 函数: $A(n)$ 的逆函数 $\alpha(n)$

正向的操作:

① 后继: $x \leftarrow x+1$

② 加法: $x+y = x+1+1+\dots+1$
y 次

③ 乘法: $x \cdot y = \underbrace{x + x + \dots + x}_{y \text{ 次}}$

④ 指数: $x^y = \underbrace{x \cdot x \cdot \dots \cdot x}_{y \text{ 次}}$

⑤ $x^{\uparrow y}$: $x^{\uparrow y} = x^{x^{\uparrow y-1}}$ $\nearrow$ y 次.

$A(m, n)$:

$m \backslash n$	0	1	2	3	...	n
0	1	2	3	4	...	$n+1$
1	2	3	4	5	...	$n+2$
2	3	5	7	9	...	$2(n+3)-2$
3	5	13	29	61	...	$2^{n+3}-3$
4	13	65533	$2^{65533}-3$	...	...	$2^{\uparrow(n+3)-3}$
5	65533	$A(4, 65533)$	...	...	...	$2^{2^{\dots 2}}$

对角线 $A(n)$. (递归 $A^{\uparrow}(n) = A(n)$)

$\Rightarrow$ 递归压栈的 MST: $O(m \cdot \underline{A(m)})$, 前缀: 排序

★ 进一步探讨的 HT:

Q: 线性时间复杂度求 MST 算法? Open.

Q: 若边权均为整数. 每个算法运行时间为 $O(1)$. $\Rightarrow O(m)$

Q: 随机算法: Karger, Klein, Tarjan. 1995. $O(n)$.

第 3/4 页: John Hopcroft, Tarjan, Dijkstra